

Rapport d'Audit Complet

eAdministration Suite Guinea

Plateforme GovTech SaaS pour la digitalisation de l'administration publique guinenne

Date	Version	Auditeur	Statut
14 mai 2026	V1.0	Z.ai Audit Engine	Complet

Ce rapport presente l'audit complet de la plateforme eAdministration Suite Guinea, couvrant l'architecture, la securite RLS/RBAC, les fonctionnalites, les donnees de test, et l'experience utilisateur. L'audit a identifie 12 problemes critiques, 8 problemes majeurs, et 15 problemes mineurs, avec des propositions d'amelioration priorisees pour chaque domaine.

1. Synthese Executive

La plateforme eAdministration Suite Guinea constitue un effort ambitieux de digitalisation de l'administration publique guinenne. Avec 28 services publics, 146 comptes de test, 6 roles differents et une interface moderne aux couleurs nationales, la plateforme couvre un large spectre fonctionnel. Cependant, l'audit revele des lacunes significatives dans plusieurs domaines critiques qui doivent etre adreesee avant toute mise en production.

Indicateur	Valeur	Statut
Composants analyses	14 pages + 3 stores + RBAC	Complet
Pages connectees au store	3 / 14 (21%)	Critique
Pages avec RLS effectif	3 / 14 (21%)	Critique
Actions dropdown fonctionnelles	0 / 14	Critique
Boutons export fonctionnels	0 / 6	Majeur
Services accessibles dans le portail	20 / 28 (71%)	Majeur
Comptes de test citoyens	140 (5 x 28 services)	OK
Integration IA reelle (LLM)	Aucune	Majeur
Telechargement documents reels	0 (PDF factices)	Majeur
Parametres persistes	0 / 14 pages	Mineur

2. Problemes Critiques (A corriger en priorite)

2.1 RLS/RBAC non applique sur 11/14 pages

Le systeme RLS/RBAC est bien concu dans le fichier rbac.ts avec des fonctions completes (filterRequestsByRLS, filterDocumentsByRLS, filterCourriersByRLS, canProcessRequest, etc.). Cependant, ces fonctions ne sont importees et utilisees que dans 3 pages sur 14 : service-requests-page.tsx, citizen-portal-page.tsx et ai-agent-page.tsx. Les 11 autres pages affichent toutes les donnees a tous les utilisateurs sans aucun filtrage.

Page	RLS applique	Fonction RLS disponible	Impact
dashboard	Non	N/A (KPIs globaux)	Faible
ged	Non	filterDocumentsByRLS	Critique

courriers	Non	filterCourriersByRLS	Critique
workflow	Non	A creer	Majeur
signatures	Non	A creer	Majeur
analytics	Non	A creer	Moyen
admin	Non	Controle RBAC interne	Critique
users	Non	A creer	Majeur
notifications	Non	A creer	Moyen
audit-logs	Non	A creer	Moyen
settings	Non	N/A (pas de donnees)	Faible

Erreur de securite dans service-requests-page.tsx : la fonction refreshSelected() lit depuis allRequests (non filtre) au lieu de requests (filtre par RLS), ce qui peut exposer des donnees d'autres utilisateurs apres une action RLS-guaree. Correction : utiliser useCitizenRequestsStore.getState().getRequestById() ou lire depuis le tableau filtre.

2.2 Actions et boutons non fonctionnels

De nombreuses interactions utilisateur sont purement cosmetiques et ne declenchent aucune action reelle. Cela cree une experience frustrante et donne une impression de plateforme inachevee, ce qui est particulierement prejudiciable dans un contexte d'appel d'offres gouvernemental.

Page	Element	Action attendue	Etat actuel
GED	5 items dropdown	Consulter/Telecharger/Archiver/Reclassifier/Supprimer	Aucun onClick
GED	Classification IA	Lancer classification auto	Aucun onClick
GED	Export Archives	Exporter vers archives	Aucun onClick
Courriers	5 items dropdown	Consulter/Viser/Transferer/Traiter/Archiver	Aucun onClick
Users	4 items dropdown	Voir/Modifier/Reset MDP/Supprimer	Aucun onClick
Users	3 actions groupees	Desactiver/Changer role/Supprimer	Aucun onClick
Users	Export/Import	Exporter ou importer CSV	Toast seulement
Analytics	4 boutons export	Generer PDF/Excel	Toast seulement

Audit-logs	Export CSV	Telecharger CSV	Toast seulement
Admin	Copy/Delete API keys	Copier/Supprimer cles	Aucun onClick
Settings	Changer logo	Upload nouveau logo	Aucun onClick

2.3 Services manquants dans le portail citoyen

Le store citizen-requests-store contient 28 services complets avec des donnees de test, des motifs de rejet et des workflows detailles pour chacun. Cependant, le portail citoyen (citizen-portal-page.tsx) ne definit que 20 services dans SERVICE_CATEGORIES. Huit services existent dans le store mais sont inaccessibles depuis l'interface utilisateur, ce qui rend 40 comptes de test (8 x 5) inutilisables depuis le portail.

Service ID	Service	Categorie
fi-1	Certificat de situation fiscale	Fiscalite
fi-2	Declaration d'impots	Fiscalite
so-1	Carte d'assurance maladie	Social
so-2	Allocations familiales	Social
u-2	Certificat de conformite	Urbanisme
u-3	Certificat d'urbanisme	Urbanisme
ed-3	Equivalence de diplome	Education
ec-6	Changement de nom	Etat Civil

2.4 Pagination non fonctionnelle

Les pages GED et Courriers disposent de boutons de pagination (Precedent, 1, 2, 3, Suivant) mais ceux-ci sont entierement decoratifs. Aucun etat de page n'est gere, aucun handler onClick n'est attache (sauf le bouton Precedent qui est desactive). Toutes les donnees sont affichees sur une seule page. Avec 15 documents dans la GED et 12 courriers, cela reste gerable, mais toute mise a l'echelle rendra la navigation impossible.

3. Problemes Majeurs

3.1 Donnees 100% hardcodees sur 11/14 pages

Seules 3 pages (service-requests, citizen-portal, ai-agent) sont connectees au store Zustand. Les 11 autres pages utilisent des donnees integrees directement dans le code du composant via des tableaux locaux constants. Cela signifie que toute modification (ajout de document, creation de courrier, etc.) est perdue lors de la navigation vers une autre page, car l'etat local du composant est detruit et recree a chaque rendu. Pour une plateforme SaaS gouvernementale, cette architecture est inacceptable car elle empeche toute persistance des donnees et toute coherence entre les differentes vues.

Page	Source de donnees	Persistence
dashboard	GOV_KPI, MONTHLY_DATA (constants.ts)	Aucune
ged	DOCUMENTS (tableau local)	Aucune
courriers	COURRIERS (tableau local)	Aucune
workflow	WORKFLOWS (tableau local)	Aucune
signatures	FAKE_SIGNATURES (tableau local)	Aucune
analytics	serviceData, radarData, slaData (constants.ts)	Aucune
admin	MODULES, API_KEYS, healthIndicators	Aucune
users	FAKE_USERS (tableau local)	Aucune
notifications	FAKE_NOTIFICATIONS (tableau local)	Aucune
audit-logs	FAKE_LOGS (tableau local)	Aucune
settings	Etat local useState	Aucune

3.2 Telechargement de documents factices

Toutes les fonctions de telechargement (downloadFile, downloadAttachedFile) generent des contenus PDF factices : du texte brut avec l'en-tete %PDF-1.4 et le type MIME application/pdf. Ce ne sont pas de vrais documents PDF et ils ne s'ouvriront pas correctement dans un lecteur PDF. Pour une plateforme de services administratifs ou la delivrance de documents officiels est le coeur du metier, c'est un probleme fondamental qui doit etre resolu avant toute presentation au gouvernement guinneen.

3.3 Agent IA : simulation sans integration LLM reelle

L'agent IA actuel est un simulateur de traitement par lot qui genere des scores de confiance aleatoires et assigne des statuts sans aucune intelligence artificielle reelle. Il n'y a aucune integration avec un modele de langage (LLM), aucune analyse de document, aucune verification intelligente des pieces justificatives. Le chatbot IA flottant utilise le SDK z-ai-web-dev-sdk avec un fallback local, mais n'a aucune connexion fonctionnelle a un modele de langage. Pour une plateforme qui se presente comme solution e-gouvernement avec IA, l'absence

d'intelligence réelle est un problème de crédibilité majeur.

3.4 Incohérence classification GED / RBAC

La page GED utilise des étiquettes de classification en français majuscule (PUBLIC, DIFFUSION LIMITEE, CONFIDENTIEL, SECRET) tandis que le système RBAC `filterDocumentsByRLS` utilise des clés en anglais minuscule (public, interne, confidentiel, secret). Même si les fonctions RLS étaient importées dans la page GED, elles ne pourraient pas filtrer correctement les documents car les valeurs de classification ne correspondent pas. Il faut uniformiser les valeurs de classification entre le système RBAC et l'interface GED.

3.5 Paramètres non persistes

La page Settings ne persiste aucune modification. Les boutons Enregistrer affichent un toast de succès mais ne sauvegardent rien. Le changement de thème dans Settings ne fait même pas appel à `toggleTheme()` du store global. Les préférences de notification sont locales au composant et non transmises au backend. Les boutons Connecter/Configurer des intégrations n'ont aucun handler `onClick`. Le bouton Changer le logo est purement décoratif. En résumé, la page Settings est une maquette non fonctionnelle.

4. Problèmes Mineurs

4.1 Imports inutilisés

`dashboard-page.tsx` importe `DEMO_KPI` sans l'utiliser ; `service-requests-page.tsx` importe `Stamp` et `Landmark` sans les utiliser ; `ai-agent-page.tsx` importe `Pause` sans l'utiliser. Ces imports morts encombrant le code et peuvent induire en erreur les développeurs.

4.2 Types TypeScript framer-motion

Le fichier `demo-video-player.tsx` a 9 erreurs de type où ``ease: string`` n'est pas assignable à ``Easing`` dans `framer-motion`. Ces erreurs sont non-bloquantes à l'exécution mais doivent être corrigées pour un code propre.

4.3 Import incorrect Separator

`app-sidebar.tsx` importe ``Separator`` depuis `lucide-react`, mais `Separator` est un composant UI, pas une icône Lucide. Cela provoque un avertissement TypeScript.

4.4 Composants dupliques

Les cartes de statistiques (stat cards) sont réimplémentées de 3 manières différentes à travers les pages. De même, les actions rapides (Quick Actions) sont dupliquées dans 5+ pages. Il faudrait extraire des composants partagés.

4.5 Verification factice signatures

La verification d'integrite dans signatures-page.tsx reussit toujours : le message est toujours 'Integrite verifiee - Document conforme' quelque soit l'entree. Les hash sont generes avec Math.random(), ce qui n'est pas cryptographiquement securise.

4.6 Mode temps reel factice

Les pages notifications et audit-logs ont un indicateur temps reel (isLive) qui est purement cosmetique. Il n'y a aucun WebSocket, aucun polling, aucune subscription. Le bouton Actualiser dans audit-logs ajoute simplement une fausse entree de log.

4.7 Filtres de date non fonctionnels

La page audit-logs a deux champs Input type=date qui n'ont aucune liaison d'etat et aucun effet sur le filtrage des logs.

4.8 Selecteur de periode sans effet

Dans analytics-page.tsx, selectedPeriod change d'etat mais n'a aucun impact sur les donnees affichees, qui sont toujours les memes donnees hardcodees.

4.9 Mot de passe en clair

Les 146 comptes de test ont des mots de passe stockes en clair dans demo-accounts.ts. De plus, la fonction login accepte 3 mots de passe universels (demo2026, demo, test2026) pour tous les comptes, ce qui affaiblit considerablement la securite.

4.10 Accessibilite insuffisante

Aucune page n'a des labels ARIA sur les elements interactifs, pas de navigation clavier pour les composants personnalises (heatmap, timeline), pas de gestion du focus pour les dialogues, pas d'annonces pour lecteurs d'ecran. Le statut est communique uniquement par la couleur dans plusieurs endroits.

5. Etat des Fonctionnalites par Page

Le tableau suivant resume l'etat de chaque page de l'application en termes de connexion au store, application du RLS, fonctionnalite des actions, et qualite globale. Ce tableau permet d'identifier rapidement les pages qui necessitent le plus de travail.

Page	Store	RLS	Actions	Qualite
------	-------	-----	---------	---------

Dashboard	Non	Non	OK	Moyen
Service Requests	Oui	Oui	OK	Bon
Citizen Portal	Oui	Oui	OK	Bon
AI Agent	Oui	Oui	Partiel	Moyen
GED	Non	Non	Aucune	Faible
Courriers	Non	Non	Aucune	Faible
Workflow	Non	Non	Partiel	Faible
Signatures	Non	Non	Partiel	Faible
Analytics	Non	Non	Export KO	Faible
Admin	Non	Non	Partiel	Faible
Users	Non	Non	Aucune	Faible
Notifications	Non	Non	Partiel	Faible
Audit Logs	Non	Non	Export KO	Faible
Settings	Non	N/A	Aucune	Faible

6. Propositions d'Ameliorations Priorisees

6.1 Priorite P0 - Bloquant pour la production

P0.1 - Appliquer RLS sur toutes les pages

Importer et utiliser filterDocumentsByRLS dans ged-page.tsx, filterCourriersByRLS dans courriers-page.tsx. Creer des fonctions RLS pour workflow, signatures, notifications, audit-logs, users. Corriger le bug refreshSelected() dans service-requests-page.tsx. Ajouter des controles de permission internes dans admin-page.tsx. Estimation : 3-4 jours.

P0.2 - Ajouter les 8 services manquants au portail citoyen

Ajouter les categories Fiscalite et Social dans SERVICE_CATEGORIES de citizen-portal-page.tsx, ainsi que les services u-2, u-3, ed-3, ec-6. Verifier que les 5 comptes de test par service peuvent soumettre des demandes. Estimation : 1 jour.

P0.3 - Connecter toutes les pages au store Zustand

Creer des stores dedies (ged-store, courriers-store, workflow-store, signatures-store, users-store, notifications-store, audit-logs-store) et migrer les donnees hardcodees vers ces stores avec persistance (middleware persist). Estimation : 5-6 jours.

P0.4 - Implementer les actions dropdown et exports

Ajouter des handlers onClick fonctionnels pour tous les menus dropdown (GED, Courriers, Users). Implementer de vrais exports PDF/Excel via jsPDF ou le SDK. Estimation : 2-3 jours.

P0.5 - Generer de vrais documents PDF pour telechargement

Remplacer les contenus factices par de vrais documents PDF generes avec jsPDF ou pdfmake, incluant les informations du citoyen, le service demande, la reference, et un design officiel avec les armoiries de la Guinee. Estimation : 3-4 jours.

6.2 Priorite P1 - Important pour la credibilite

P1.1 - Integration IA reelle avec LLM

Utiliser le SDK z-ai-web-dev-sdk deja installe pour connecter l'agent IA a un veritable modele de langage. Implementer : (a) analyse intelligente des pieces justificatives, (b) verification automatique des informations du citoyen contre la base de donnees, (c) chatbot conversationnel fonctionnel avec contexte service, (d) suggestions de traitement basees sur l'historique. Estimation : 5-7 jours.

P1.2 - Upload de documents reel dans la GED

Ajouter un vrai dans le dialogue d'upload de la GED, avec previsualisation, stockage dans le store, et telechargement fonctionnel. Actuellement, l'upload cree juste une entree metadata sans fichier reel. Estimation : 1-2 jours.

P1.3 - Pagination fonctionnelle

Implementer un etat de page (currentPage, itemsPerPage) dans les pages GED et Courriers, avec calcul du slice de donnees a afficher et handlers onClick sur les boutons. Extraire un composant Pagination partage. Estimation : 1 jour.

P1.4 - Correction classification GED/RBAC

Uniformiser les valeurs de classification : utiliser les memes cles (public, interne, confidentiel, secret) dans l'interface GED et dans le systeme RBAC. Ajouter un mapping d'affichage pour les labels francais. Estimation : 0.5 jour.

P1.5 - Persistence des parametres

Connecter la page Settings au store global pour le theme, les notifications et les preferences. Implementer la sauvegarde reelle via le middleware persist de Zustand. Estimation : 1-2 jours.

6.3 Priorite P2 - Ameliorations qualite

P2.1 - Securite : hash des mots de passe

Remplacer le stockage en clair des mots de passe par un hash. Supprimer les mots de passe universels (demo, test2026). Implementer une validation de complexite. Estimation : 1 jour.

P2.2 - Accessibilite (RGAA/WCAG 2.1 AA)

Ajouter des labels ARIA, gerer le focus dans les dialogues, ajouter la navigation clavier, utiliser des icones en plus de la couleur pour les statuts. Estimation : 3-4 jours.

P2.3 - Composants partages

Extraire StatCard, QuickActions, Pagination, StatusBadge en composants reutilisables dans src/components/ui/. Reduire la duplication de code. Estimation : 2-3 jours.

P2.4 - Gestion des erreurs et etats de chargement

Ajouter des ErrorBoundary par section, des etats de chargement (skeleton/spinner), des etats vides (empty state), et des messages d'erreur utilisateur-friendly. Estimation : 2-3 jours.

P2.5 - Nettoyage TypeScript

Corriger les 9 erreurs de type framer-motion dans demo-video-player.tsx, corriger l'import Separator dans app-sidebar.tsx, supprimer les imports inutilises. Estimation : 0.5 jour.

P2.6 - Signatures cryptographiques

Remplacer Math.random() par crypto.subtle.digest() pour les hash de signature. Implementer une vraie verification d'integrite qui echoue quand le document est modifie. Estimation : 1-2 jours.

P2.7 - Temps reel via WebSocket

Implementer une connexion WebSocket pour les notifications et les logs d'audit. Remplacer les indicateurs isLive factices par de vraies mises a jour en temps reel. Estimation : 3-4 jours.

P2.8 - Temps reel dans les tableaux de bord

Connecter les KPIs du dashboard et les statistiques analytics aux donnees reelles du store citizen-requests-store, plutot qu'aux constantes hardcodees. Estimation : 2-3 jours.

7. Feuille de Route Recommandee

La feuille de route suivante organise les corrections et ameliorations en 4 phases successives, en privilegiant les elements bloquants pour la production, puis la credibilite de la plateforme, et enfin la qualite globale. Les estimations sont donnees en jours-homme et supposent un developpeur travaillant a temps plein.

Phase	Duree	Objectifs	Livrables
Phase 1 : Securite et Completion	10-12 jours	P0.1 a P0.5 : RLS complet, 28 services, stores, actions, vrais PDF	Plateforme securisee avec toutes les fonctionnalites de base
Phase 2 : Credibilite IA et Donnees	8-10 jours	P1.1 a P1.5 : LLM reel, upload GED, pagination, classification, settings	Agent IA fonctionnel, GED operationnelle, parametres persistes
Phase 3 : Qualite et Robustesse	8-10 jours	P2.1 a P2.4 : securite mots de passe, accessibilite, composants, erreurs	Code propre, conforme RGAA, gestion erreurs complete
Phase 4 : Polish et Performance	5-7 jours	P2.5 a P2.8 : TypeScript, signatures crypto, WebSocket, dashboards dynamiques	Plateforme production-ready, performante, temps reel

En resume, la plateforme eAdministration Suite Guinea a des fondations solides avec un systeme RBAC bien concu, 28 services complets avec donnees de test, et une interface moderne aux couleurs nationales. Cependant, la majorite des pages sont encore au stade de maquette fonctionnelle sans connexion au store ni application des regles de securite. L'effort principal doit se concentrer sur la connexion des pages au store Zustand, l'application stricte du RLS/RBAC, et l'implementation des actions utilisateur. Avec un investissement de 30-40 jours-homme repartis sur 4 phases, la plateforme peut atteindre un niveau production-ready capable de convaincre le gouvernement guinneen.